

Simple MATSim replanning recipe

Fadi El Eid¹, Sonja Fessler², Johannes Müller³, Denissa Purba³,
Franz-Xaver Rupprecht³, and Gunnar Flötteröd^{4,5,*}

¹ TU München, ² WU Wien, ³ BOKU University, ⁴ Linköping University,

⁵ Swedish National Road and Transport Research Institute, * corresponding author

The authors of this document are a self-selection-biased sample of participants in the Linköping/BOKU-organized course “Introduction to agent-based modeling”.

Introduction

The “MATSim loop” repeatedly executes the following steps: Replanning (each agent revisits its travel plan choice set and decides which plan to execute), mobility simulation (all agents execute their plans), and scoring (each agent evaluates the most recently executed plan). When replanning, each agent selects and applies a *replanning strategy*. This can be an *innovation* strategy or a *selection* strategy (Horni et al., 2016). After many iterations, the simulation usually stabilizes, which then is taken as an approximation of the long-term behavior of the real system under consideration.

Newly innovated plans need to be executed in the mobility simulation in order to receive a score. Executing nonsensical plan innovations hence impairs the overall realism of the simulation. The *Discrete Mode Choice* contrib (Hörl and Balac, 2026; Hörl et al., 2019) addresses this by building an anticipation of the (mode-specific) score of a plan before executing it. The *Pseudo Simulation* contrib (Fourier, 2026; Fourie, 2016) executes plans given fixed travel times to anticipate their scores before executing them in the network. The *ATAP* contrib (Flötteröd, 2026a,b) approximates noise-free point equilibria and relies on a plan emulation similar to that of *Pseudo Simulation* to only execute plans that likely increase the score. These contribs have in common that they modify the original replanning model by adding a score anticipation, which is in principle a reasonable thing to do (Sandholm, 2015).

We consider here the original MATSim setup without score anticipation, where several replanning strategies are given and, in each iteration, each agent applies a strategy according to some given strategy selection probability distribution. We devise a simple and broadly applicable strategy weighting schedule that aims to approximate system states in which every agent eventually executes only near-optimal plans.

Greedy search without selection

No score uncertainty

To fix thoughts, assume that the score received by an executed plan is an exact predictor of the score obtained when re-executing the plan. This idealized setting requires (i) no stochasticity in the mobility simulation, and (ii) no relevant changes in the simulated environment (mobsim) because of other agents' plan switching.

Given this assumption, consider an agent n that only uses innovation strategies and the *WorstPlanSelector* for plan removal. Index the agent's J plans by increasing score $S_n^{(1)} \leq \dots \leq S_n^{(J)}$. Let S_n^{inno} be the score of a newly innovated plan. It replaces the plan with score $S_n^{(1)}$ if $S_n^{(1)} < S_n^{\text{inno}}$; otherwise, it is discarded. The score sum over all J plans hence changes by

$$\max \left\{ 0, S_n^{\text{inno}} - S_n^{(1)} \right\} \geq 0$$

from one iteration to the next. Scores will no longer change once $S_n^{(1)}$ equals the largest possible realization $\hat{S}_n^{\text{inno}}$ of S_n^{inno} . Due to the ordering of plan scores and the impossibility of exceeding $\hat{S}_n^{\text{inno}}$, this will result in

$$S_n^{(1)} = \dots = S_n^{(J)} = \hat{S}_n^{\text{inno}} \quad \text{after many iterations.}$$

This equality holds only for plans in the choice set. Plans with scores below $\hat{S}_n^{\text{inno}}$ may still be innovated and executed before being discarded again. Still, if one is only interested in finding subjectively optimal plans per agent, the choice set will eventually contain only such plans.

Given no score uncertainty, an innovation-only replanning with *Remove-WorstPlan* implements a greedy search for all non-zero choice set sizes.

With score uncertainty

Given stochastic plan scores, a newly innovated plan may replace the currently worst plan because it has a *by chance* higher score. The result are upwards-biased scores in the choice set. Since the considered innovation-only setting never re-evaluates the scores of plans in the choice set, the overall score in that set grows towards an unrealistically high value. This makes it increasingly difficult for relevant plan innovations to enter the choice set; the search can be expected to stagnate.

This stagnation can be reduced if plans in the choice set are occasionally re-executed. Recalling that the considered setup has a tendency to create a choice set of plans with maximal and hence equal scores, the added value of selecting plans for re-execution other than uniformly is not obvious. We hence add *SelectRandom* to the strategy mix.

Given a plan choice set size of J and a probability α of using any innovation strategy, the selection probability of any concrete plan in the choice set is $(1 - \alpha)/J$,

meaning that it is re-evaluated on average every $J/(1 - \alpha)$ iterations. We require that each plan in the choice set is on average re-evaluated once between two consecutive innovations because then the comparison of a newly innovated plan to the choice set is, on average, not affected by any persistent score bias in that set. This requirement translates into

$$\alpha \leq \frac{1}{J+1}.$$

An assignment can be expected to make the initially largest process with a maximum of exploration, which is achieved by the smallest possible choice set: $J = 1$, allowing for $\alpha = 0.5$.

Given a stochastic setting that requires to re-evaluate the plan choice set on average once between plan innovations, the initially fastest assignment process can be expected from using a plan choice set of size one and a 50 % innovation and selection rate, respectively.

Reducing score uncertainty

Occasionally re-evaluating plans in the choice set keeps scores from growing unrealistically large, but it does not prevent newly innovated plans that only by chance have received high scores from entering the choice set. The magnitude of this bias can be reduced by reducing score uncertainty.

Stochasticity in the mobility simulation is conceptually not a major concern because (i) the mobility simulation can be configured to (near-)determinism, and (ii) stochasticity can be controlled by running multiple independent mobsims and averaging scores. Thus, we ignore this source of uncertainty.

Score uncertainty due to variable network conditions, even in a deterministic mobility simulation, can be controlled by gradually reducing the number of agents that get to switch their plan simultaneously (“cooling”). This amounts to gradually reducing α to zero over the course of the assignment iterations. The Robbins and Monro (1951) conditions may be adopted here: If α is to fall with one over a power of the iteration number k , i.e. $\alpha(k) = 1/(k+1)^p$, these conditions require $p \in (0.5, 1]$. The larger p , the faster the cooling.

Figure 1 illustrates two different search situations, where the assignment problem is visualized by the contour lines of a two-dimensional minimization problem. The search on the left takes precise steps towards the minimum in the center, but too large steps; causing the search to repeatedly overshoot. The search on the right takes smaller steps and less precise steps; thus, it wanders around the minimum. The overshooting search leads to rapid yet imprecise coverage of the region around the minimum. Consequently, averaging just a handful of visited points in the overshooting search can be expected to approximate the center point quite well. In contrast, averaging visited points in the random search can be expected to approach the center much more slowly, as the search tends to remain on the same side of the center for many iterations.

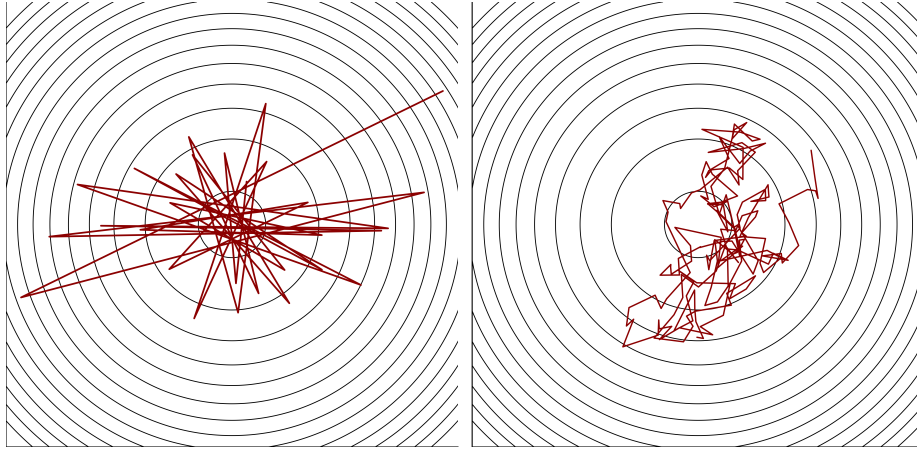


Figure 1: Overshooting (left) vs. random (right) search

The precise (overshooting) search direction considered here corresponds to a plan innovation that attempts to optimize an aspect of the new plan given the current network conditions. The *ReRoute* strategy does this. On the opposite, a random and less (or un-) directed search is implemented in *SubtourModeChoice* and *TimeAllocationMutator*. Since *SubtourModeChoice* only considers a handful of modes while *TimeAllocationMutator* deals with a continuum of departure times, one may assume that *SubtourModeChoice* is still much less inefficient than *TimeAllocationMutator*. This is also confirmed by empirical evidence.

Cooling should only start after near-stationary conditions have been reached.

- If at least one inefficient innovation strategy is used, in particular *TimeAllocationMutator*, one may use the innovation schedule $\alpha(k) = 1/\sqrt{k+1}$. It cools the system down at the slowest possible rate that is still (borderline-)compatible with the Robbins and Monro (1951) conditions.
- If only efficient innovation strategies are used, in particular *ReRoute*, one may use the innovation schedule $\alpha(k) = 1/(k+1)$. It cools the system down at the fastest possible rate that is still compatible with the Robbins and Monro (1951) conditions.
- *SubtourModeChoice* may be treated as a relatively efficient innovation strategy (and be cooled down quickly) because it needs to explore just a handful of transport modes.

Recipe

Building on these discussions, we suggest the following two-stage assignment logic.

Overall settings.

- Use only innovation strategies and *SelectRandom*.
- Do not “turn off” any strategies.
- Limit the plan choice set size to one.
- Use *WorstPlanSelector* for plan removal.

First stage. In each iteration, use a random innovation strategy with probability 0.5. Otherwise use *SelectRandom*, i.e., re-evaluate the single memorized plan. Run this stage until near-stationarity is attained.

Second stage. Start with the final plans file from the first stage. With k the iteration index starting from zero in Stage 2, use a random innovation strategy with probability $\alpha(k)$ and *SelectRandom* with probability $1 - \alpha(k)$ where

$$\alpha(k) = \begin{cases} \frac{1}{2} \frac{1}{\sqrt{k+1}} & \text{if at least one slow innovation strategy} \\ \frac{1}{2} \frac{1}{k+1} & \text{if only fast innovation strategies.} \end{cases}$$

The factor of $1/2$ in the second stage innovation probability ensures that innovation changes continuously from Stage 1 to Stage 2. Overall, this resembles the *sigmoid* cooling schedule also available in the *ReplanningAnnealer* default functionality. A difference is that the cooling phase reduces innovation more slowly than the *sigmoid* option, which has an exponential tail that is not compatible with Robbins and Monro (1951).

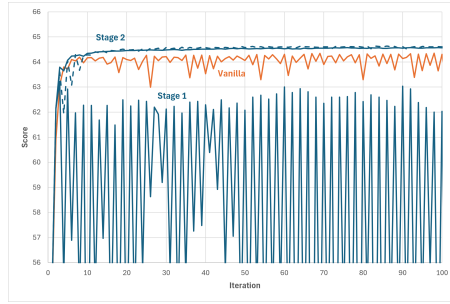
The analysis motivating this recipe was more intuitive than rigorous. Still, within the default MATSim configuration options, it is not obvious how a more complicated setup could approach less biased stationary conditions at a higher rate.

Averaging scores over iterations to filter out stochasticity may be a useful complementary measure, even though it complexifies the setup by (i) requiring to define the filter strength, and (ii) introducing an additional inertia in the assignment process.

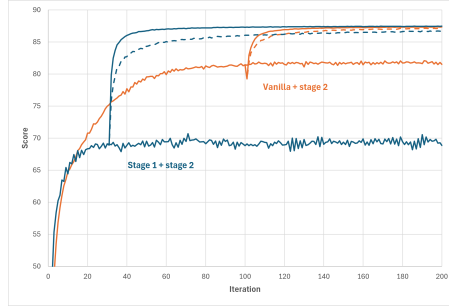
Results

We consider four scenarios for the cities of Berlin (Ziemke et al., 2015), Munich (Moeckel et al., 2020), Stockholm (Flötteröd, 2026a), and Oslo (Flügel et al., 2024, 2023).

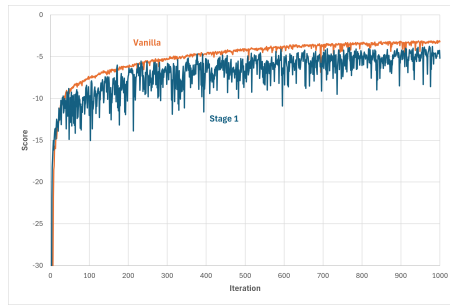
Berlin uses only *ReRoute* for innovation. Assignment results (population-average executed scores over iterations) are shown in Figure 2a. The Vanilla run uses *ReRoute* with constant probability 0.1 and *ChangeExpBeta* otherwise, with five plans per agent. The Stage 1 run oscillates almost immediately, such that Stage 2 is run from the beginning on (no continuation from Stage 1). The solid Stage 2 line is obtained by *fast*



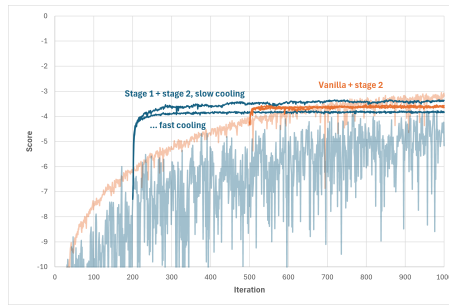
(a) Berlin



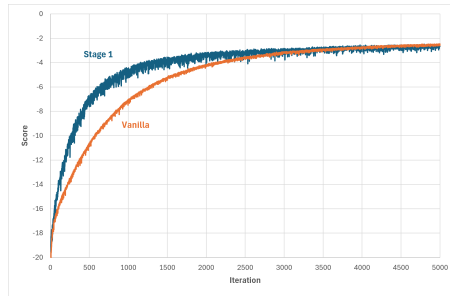
(b) Munich



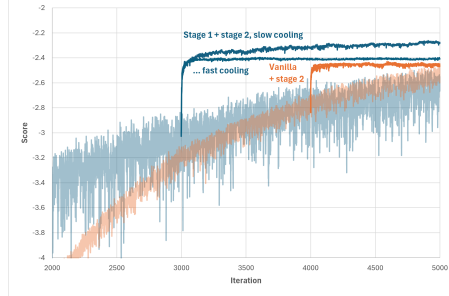
(c) Stockholm



(d) Stockholm (enlarged)



(e) Oslo



(f) Oslo (enlarged)

Figure 2: Experimental results

cooling (one over iteration number), the dashed line by *slow cooling* (one over square root of iteration number). Fast cooling stabilizes most quickly and attains largest score values. Although average score maximization is not necessarily equivalent to equilibration (individual score maximization), we use it here as a measure of individual agent satisfaction. These results support the proposed recipe, and that fast cooling is adequate if only fast innovation strategies are present.

Munich uses *ReRoute* and *SubtourModeChoice*. Results are shown in Figure 2b, using the same layout as for Berlin. The vanilla run uses four plans per agent, *ChangeExpBeta* with probability 0.8 and *ReRoute* and *SubtourModeChoice* with probability 0.1, respectively. Stage 1 stabilizes before Vanilla, but at a much lower score. This can be explained by the more aggressive (less precise) innovation of Stage 1. Both experiments are continued from near-stabilization on with a Stage 2 run (Stage 1 is continued from iteration 30 on; vanilla is continued from iteration 100 with an initial innovation rate scaled to 0.2 for continuity). All Stage 2 runs attain comparable score levels. Fast cooling outperforms slow cooling. This supports the hypothesis that *SubtourModeChoice* does not warrant a slow cooling. Best progress is obtained by the proposed recipe.

Stockholm uses *ReRoute* and *TimeAllocationMutator*. Figure 2c indicates an overall much slower score stabilization, which can be explained the inefficient *TimeAllocationMutator*. Vanilla uses five plans per agent, *ChangeExpBeta* with probability 0.8 and *ReRoute* and *TimeAllocationMutator* with probability 0.1, respectively. Figure 2d zooms in along the score axis and also includes Stage 2 results. The half-transparent lines are the zoomed-in versions of those in Figure 2c. Stage 1 is arbitrarily cooled down earlier (in iteration 200) than Vanilla (in iteration 500) to illustrate that a precise stabilization of Stage 1 is not necessary. The slow and fast cooling line of the continued vanilla run are visually indiscernible. Although scores keep drifting upwards, the proposed recipe with slow cooling performs overall best. Slow cooling appears adequate to follow the slow *TimeAllocationMutator* progress.

Oslo uses *ReRoute*, *TimeAllocationMutator*, and *SubtourModeChoice*. The results shown in Figures 2e and 2f convey qualitatively the same message as for Stockholm: The proposed recipe performs best; the *TimeAllocationMutator* warrants slow cooling.

References

- S. Flügel, G. Flötteröd, R. Ringdal, and C. Weber. Tramodsim: An open-source traffic assignment tool for tramod_by based on matsim. Technical Report 1993/2023, Norwegian Centre for Transport Research, 2023. URL <https://www.toi.no/publikasjoner>.
- S. Flügel, C. Weber, S.S. Klommestein, J. Korsmo, and A. Kielland. Towards activity-based demand modeling for the greater oslo area: Using machine learning to predict travel mode and activity plans. Technical Report 2065/2024, Norwegian Centre for Transport Research, 2024. URL <https://www.toi.no/publikasjoner>.
- G. Flötteröd. A simulation heuristic for traveler- and vehicle-discrete dynamic traffic assignment. *Transportation Research Part B: Methodological*, 206:

- 103406, 2026a. ISSN 0191-2615. doi: <https://doi.org/10.1016/j.trb.2026.103406>. URL <https://www.sciencedirect.com/science/article/pii/S0191261526000184>.
- G. Flötteröd. Matsim atap contrib, 2026b. URL <https://github.com/matsim-org/matsim-libs/tree/main/contribs/atap>. Accessed: 2026-05-12.
- P.J. Fourie. Multi-modeling in matsim: Psim. In Andreas Horni, Kai Nagel, and Kay W. Axhausen, editors, *The Multi-Agent Transport Simulation MATSim*, pages 263–266. Ubiquity Press, London, 2016. doi: <http://dx.doi.org/10.5334/baw.39>. License: CC-BY 4.0.
- P. Fourier. Matsim pseudo-simulation contrib, 2026. URL <https://github.com/matsim-org/matsim-libs/tree/main/contribs/pseudosimulation>. Accessed: 2026-05-12.
- A. Horni, K. Nagel, and K.W. Axhausen. Plan selection (choice). In *The Multi-Agent Transport Simulation MATSim*, chapter 4.6.3, pages 40–41. Ubiquity Press, London, 2016. doi: <http://dx.doi.org/10.5334/baw>. License: CC-BY 4.0.
- S. Hörl and M. Balac. Matsim discrete mode choice contrib, 2026. URL https://github.com/matsim-org/matsim-libs/tree/main/contribs/discrete_mode_choice. Accessed: 2026-05-12.
- S. Hörl, M. Balac, and K.W. Axhausen. Pairing discrete mode choice models and agent-based transport simulation with matsim. In *Proceedings of the 98th Annual Meeting of the Transportation Research Board*, Washington, D.C., January 2019.
- Rolf Moeckel, Nico Kuehnel, Carlos Llorca, Ana Tsui Moreno, and Hema Rayaprolu. Agent-based simulation to improve policy sensitivity of trip-based models. *Journal of Advanced Transportation*, 2020(1):1902162, 2020. doi: <https://doi.org/10.1155/2020/1902162>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1155/2020/1902162>.
- H. Robbins and S. Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.
- William H Sandholm. Population games and deterministic evolutionary dynamics. In *Handbook of game theory with economic applications*, volume 4, pages 703–778. Elsevier, 2015.
- Dominik Ziemke, Kai Nagel, and Chandra Bhat. Integrating CEMDAP and MATSIM to increase the transferability of transport demand models. *Transportation Research Record: Journal of the Transportation Research Board*, 2493(1):117–125, 2015. doi: 10.3141/2493-13. URL <https://doi.org/10.3141/2493-13>.